Übung 3 - Schleifen und Funktionen

Programmiervorkurs 2026

1 Schleifen

1.1 Summe

In dieser Aufgabe soll das Berechnen der Summe der Zahlen 1 bis 20 (eingeschlossen) auf unterschiedlichen Wegen implementiert werden.

1.1.1 Gaußsche Summenformel

Um eine Referenz zu haben und mit einem einfachen Beispiel zu beginnen, soll zuerst Berechnung einer Summe mithilfe der Gaußschen Summenformel implementiert werden.

$$1 + 2 + 3 + \ldots + n = \sum_{k=1}^{n} k = \frac{n \cdot (n+1)}{2}$$

1.1.2 while-Schleife

Da nicht alle Zahlenreihen einfach aufsteigend und damit mit der Gaußschen Summenformel berechenbar sind, soll nun die Summe unter Verwendung einer **while**-Schleife implementiert werden.

1.1.3 for-Schleife

Abschließend soll die Summe mithilfe einer **for**-Schleife umgesetzt werden.

1.2 Fakultät

1. Schreibe ein Programm, das den Wert des Ausdrucks $1 * 2 * 3 * \ldots * 15 = 15!$ (Fakultät von 15) berechnet und das Ergebnis auf der Konsole ausgibt.
2. Erweitere dein Programm, sodass es von beliebigen Eingaben in der Konsole die Fakultät berechnet.

1.3 Listenverarbeitung

Häufig gibt es in Programmen eine Variable, in der das Ergebnis langsam aufgebaut wird. Beispielsweise ist diese für eine Summe zu Anfang 0. Danach wird diese in einer Schleife für jeden Eintrag verändert. Schreibe ein Programm, das die für die Liste `[10, 2, 7, 4, 3, 5, 8]` die folgenden Werte berechnet:

1. Die Summe aller Elemente
2. Das Produkt aller Elemente
3. Der Durchschnitt aller Elemente
4. Eine Liste der Quadrate der Elemente

Was musst du bei der Berechnung des Durchschnitts beachten?
Schaffst du es, die Aufgabe mit nur einer Schleife zu lösen?

1.4 Primzahl

In dieser Aufgabe soll ein Programm zur Erkennung von Primzahlen geschrieben werden. Lasse dein Programm dafür eine Zahl mittels `input()` einlesen und überprüfen, ob diese eine Primzahl ist. Teste dein Programm mit verschiedenen Werten.

2 Dateien

In dieser Aufgabe soll der Umgang mit Dateien beigebracht werden, d.h. das Schreiben und Lesen von Text.

2.1 Daten schreiben

Info

Bitte vergewissert euch, dass der Pfad korrekt ist und ihr keine existierenden Dateien überschreibt, die euch wichtig sind!

Folgende Daten einer fiktiven Programmiervorkursanmeldung sollen im Format ”comma-separated values” (kurz CSV)[1] in eine Datei geschrieben werden:

```
bestellungen = [
    ('joern.siegmund@gmail.com', 'Jörn Siegmund', 'Informatik'),
    ('alena_joerg@aol.com', 'Alena Jörg', 'Elektrotechnik'),
    ('gema76@yahoo.com', 'Gerta Volkmar', 'Informatik'),
    ('lia.wendeltreppe@proton.me', 'Lia Wendel', 'Cognitive Science'),
    ('leon.heinrich@gmail.com', 'Leon Heinrich', 'Wirtschaftsinformatik'),
    ('harter-mut1996@gmx.de', 'Christoph Hartmut', 'Maschinenbau')
]
```

Erstellt dazu mithilfe von Python eine neue Datei (mit Dateiendung `.csv`) und schreibt die obigen Werte in die erstellte Datei. Existiert die Datei bereits, sollen die Daten einfach überschrieben werden bzw. am Dateianfang begonnen werden. Jedes Tuple soll dabei in eine eigene Zeile, wobei die einzelnen Einträge des Tuples mit Kommata separiert sein sollen. Die erste Zeile sollen die Spaltenüberschriften ”Mail”, ”Name” und ”Studiengang” haben.

Überprüft nach der Ausführung im Texteditor eures Vertrauens, ob die Werte korrekt geschrieben wurden. Macht euch außerdem Gedanken, bei welchen Nutzereingaben eventuell Probleme mit dem verwendeten Format auftreten könnten.

2.2 Daten lesen

In der vorherigen Teilaufgabe habt ihr (hoffentlich erfolgreich) Dateien erstellt und in diese geschrieben. In dieser Aufgabe sollen nun diese Daten wieder eingelesen werden. Öffnet dazu die in der letzten Teilaufgabe erstellte Datei in Python und lest alle Zeilen als Liste von Strings ein. Zählt, wie häufig einzelne Studiengänge angegeben wurden und gebt diese in der Konsole aus.

[1] Wikipedia-Artikel zu CSV

3 Funktionen

3.1 Einstieg

1. Als Einstieg soll eine Hilfsfunktionen zum Umrechnen von Einheiten definiert werden, im Genaueren Zoll $\rightarrow$ cm.
2. Schreibe nun eine Funktion, welche eine Liste von Zahlen als Parameter nimmt, das größte Element in dieser bestimmt und es zusammen mit seinem Index in der Liste auf der Konsole ausgibt. Achte dabei auf Sonderfälle.

3.2 Bildschirmdiagonale

Nun soll die Größe eines Bildschirms im metrischen System anhand des Seitenverhältnisses und der Bildschirmdiagonale in Zoll berechnet werden. Schreibe dafür eine Funktion, welche die Bildschirmdiagonale in Zoll und das Seitenverhältnis (Breite:Höhe) entgegennimmt und die Breite und Höhe in Zentimetern zurückgibt.
Hinweis: Es kann die Konvertierungsfunktion aus der vorherigen Aufgabe wiederverwendet werden.
Die mathematischen Zusammenhänge sind in Abbildung 1 illustriert.

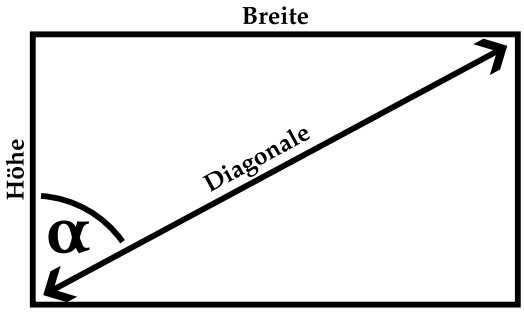


Abbildung 1: Mathematische Zusammenhänge der Bildschirmdiagonale

Hinweis: Sinus, Kosinus und Tangens sind mit `math.sin()`, `math.cos()`, `math.tan()` *verfügbar bzw.* `math.asin()`, `math.acos()` *und* `math.atan()` *für Arcus Sinus, etc.. Diese Funktionen können genutzt werden, nachdem sie importiert wurden. Beispiel:*

```
import math

radians = math.pi * 0.5
value = math.sin(radians) # => 1.0
```

3.3 Farbkonvertierung (Bonus)

Abschließend soll noch die Konvertierung von Farben aus dem HSV- in den RGB-Farbraum implementiert werden (mehr Infos auf Wikipedia). Die Werte einer Farbe im HSV-Format (**H**ue, **S**aturation, **V**alue) werden in den Wertebereichen $H \in [0, 360]$, $S \in [0, 1]$ und $V \in [0, 1]$ angegeben. Die Verwendung des HSV-Formats wird zum Beispiel für die Generierung von Farben mit gleichem Farbtonabstand benötigt.

Schreibe eine Funktion, welche die Parameter h, s und v hat und ein Dreier-Tuple für die RGB-Farben zurückgibt.

$$h_i = \left\lfloor \frac{H}{60°} \right\rfloor$$

$$f = \frac{H}{60°} - h_i$$

$$\begin{aligned} p &= V \cdot (1 - S) \\ q &= V \cdot (1 - S \cdot f) \\ t &= V \cdot (1 - S \cdot (1 - f)) \end{aligned}$$

$$(r, g, b) = \begin{cases} (q, V, p) & h_i = 1 \\ (p, V, t) & h_i = 2 \\ (p, q, V) & h_i = 3 \\ (t, p, V) & h_i = 4 \\ (V, p, q) & h_i = 5 \\ (V, t, p) & sonst \end{cases} \tag{1}$$

Die resultierenden RGB-Farben befinden sich im Wertebereich [0,1].
Hinweis: Mit der Funktion math.floor() können Fließkommazahlen abgerundet werden.